

## Contents

|                                                             |    |
|-------------------------------------------------------------|----|
| Plain Java .....                                            | 3  |
| College class (simple POJO).....                            | 3  |
| Main class (YOU manage everything) .....                    | 3  |
| No Spring means: .....                                      | 3  |
| Coupling .....                                              | 4  |
| Factory-style Java .....                                    | 4  |
| Why plain Java does NOT scale well .....                    | 6  |
| Interview .....                                             | 6  |
| Spring framework.....                                       | 7  |
| Example.....                                                | 7  |
| package .....                                               | 7  |
| Import Statements .....                                     | 7  |
| Class declaration .....                                     | 8  |
| main method .....                                           | 8  |
| Creating Spring Container .....                             | 8  |
| Getting a bean from Spring.....                             | 9  |
| Calling business logic.....                                 | 9  |
| Execution flow .....                                        | 9  |
| Spring Boot.....                                            | 10 |
| Your CURRENT code (Plain Spring).....                       | 10 |
| Above application using Spring Boot .....                   | 10 |
| Main Class .....                                            | 10 |
| College class (almost same) .....                           | 11 |
| What changed? .....                                         | 11 |
| What <code>@SpringBootApplication</code> really doing?..... | 11 |
| BENEFITS of Spring Boot .....                               | 12 |
| How to EXECUTE Spring Boot app.....                         | 13 |
| Project structure (Spring Boot way).....                    | 13 |
| Interview .....                                             | 13 |
| When should YOU use Spring Boot? .....                      | 13 |
| REST API .....                                              | 14 |
| Create a Spring Boot project.....                           | 14 |
| Required dependency .....                                   | 14 |
| Project structure (IMPORTANT) .....                         | 14 |

|                                                           |    |
|-----------------------------------------------------------|----|
| Main Application class .....                              | 15 |
| Business logic (same as before) .....                     | 15 |
| REST Controller (NEW) .....                               | 16 |
| What annotations mean (VERY IMPORTANT).....               | 16 |
| How execution works (FLOW).....                           | 16 |
| How to RUN the application .....                          | 17 |
| How to TEST the REST API.....                             | 17 |
| Why REST API is better than <code>main()</code> call..... | 17 |
| Interview one-liner.....                                  | 17 |
| What next.....                                            | 18 |

## Plain Java

### College class (simple POJO)

```
package com.testing.qe;

public class College {

    public void principalName() {
        System.out.println("Principal is Dr. Sharma");
    }
}
```

If you look at above Java code

- No annotations
- No framework
- Just a normal Java class

### Main class (YOU manage everything)

```
package com.testing.qe;

public class TestApplication {

    public static void main(String[] args) {

        // YOU create the object
        College college = new College();

        // YOU call the method
        college.principalName();
    }
}
```

### No Spring means:

| Responsibility        | Who does it now? |
|-----------------------|------------------|
| Object creation       | YOU              |
| Dependency management | YOU              |
| Lifecycle control     | YOU              |
| Configuration         | YOU              |
| Scalability support   | ×                |
| Loose coupling        | ×                |

## Coupling

### With out Spring : Tight coupling

```
public class College {  
    private Principal principal = new Principal(); // hard-coded  
    public void showPrincipal() {  
        principal.name();  
    }  
}
```

Problems:

- Cannot replace `Principal`
- Hard to test
- Code changes needed everywhere

### With Spring : Loose coupling

```
@Component  
public class College {  
    @Autowired  
    private Principal principal;  
}
```

Spring:

- Creates `Principal`
- Injects it
- Allows easy replacement

## Factory-style Java

Before Spring existed, people did this

```
public class ObjectFactory {  
  
    public static College getCollege() {  
        return new College();  
    }  
}
```

Usage:

```
College college = ObjectFactory.getCollege();
```

Spring is basically a **VERY powerful, configurable factory**



## Why plain Java does NOT scale well

Imagine:

- 50 classes
- Each depends on 5 others

Without Spring:

- 250+ `new` statements
- Tight coupling
- Nightmare to maintain

With Spring:

- Zero `new`
- Configuration-based wiring

## Interview

- ✓ Small utility programs
- ✓ Simple learning demos
- ✓ One-class applications
- ✗ Enterprise apps
- ✗ REST APIs
- ✗ Database-heavy systems

# Spring framework

## Example

```
package com.testing.qe;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Test {
    public static void main(String[] args) {
        ApplicationContext context = new AnnotationConfigApplicationContext(CollegeConfig.class);
        College college = context.getBean("college", College.class);
        college.principalName();
    }
}
```

## package

package com.testing.qe;

- This defines the **package (namespace)** of the class.
- It helps:
  - Organize code
  - Avoid class name conflicts
  - Enable **component scanning** in Spring

## Import Statements

import org.springframework.context.ApplicationContext;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;

| ApplicationContext                                                                                                                                                                                                                                                                                                      | AnnotationConfigApplicationContext                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Central <b>Spring IoC container</b></li><li>• Responsible for:<ul style="list-style-type: none"><li>○ Creating objects (beans)</li><li>○ Managing their lifecycle</li><li>○ Injecting dependencies</li></ul></li></ul> <p>Think of it as <b>Spring's factory + manager</b>.</p> | <ul style="list-style-type: none"><li>• A <b>concrete implementation</b> of <code>ApplicationContext</code></li><li>• Used when:<ul style="list-style-type: none"><li>○ Configuration is done using <b>annotations</b></li><li>○ No <code>applicationContext.xml</code> is used</li></ul></li></ul> <p>This class:</p> <ul style="list-style-type: none"><li>• Reads <code>@Configuration</code></li><li>• Scans <code>@Component</code></li><li>• Handles <code>@Bean</code>, <code>@Autowired</code>, etc.</li></ul> |

## Class declaration

```
public class Test {
```

This is a **plain Java class**, not managed by Spring.

⚠️ Important:

- `Test` itself is **not a Spring bean**
- It just **boots the Spring container**

## main method

```
public static void main(String[] args) {
```

- Java application entry point
- Starts Spring manually (not Spring Boot)

## Creating Spring Container

```
ApplicationContext context = new AnnotationConfigApplicationContext(CollegeConfig.class);
```

**What happens here step-by-step:**

1. Spring container is created
2. `CollegeConfig.class` is loaded

```
@Configuration

@ComponentScan("com.testing.qe")

public class CollegeConfig {

}
```

3. Spring:

- Reads `@Configuration`
- Scans packages for:
  - `@Component`
  - `@Service`
  - `@Repository`
  - `@Controller`
- Creates beans

At this moment, all beans are created and stored in the container.



## Getting a bean from Spring

```
College college = context.getBean("college", College.class);
```

### What this means:

- "college" → **bean ID**
- `College.class` → expected type

Spring:

1. Looks for a bean named **college**
2. Confirms it is of type `College`
3. Returns the object

Usually defined as:

```
@Component
public class College {
    public void principalName() {
        System.out.println("Principal is Dr. Sharma");
    }
}
```

Bean name defaults to **class name with first letter lowercase**.

## Calling business logic

```
college.principalName();
```

- Calls method on Spring-managed object
- Any dependencies inside `College` are:
  - Already injected
  - Already initialized

## Execution flow

JVM starts

↓

main() method executes

↓

Spring container starts

↓

Configuration class loaded

↓

Beans created & wired

↓

getBean() fetches object

↓

Method executed

## Spring Boot

Compare **your above Spring code** with **Spring Boot**, then we will cover **benefits** and **how to execute**.

### Your CURRENT code (Plain Spring)

```
ApplicationContext context = new AnnotationConfigApplicationContext(CollegeConfig.class);
```

```
College college = context.getBean("college", College.class);
```

```
college.principalName();
```

#### What YOU do manually

- Create `ApplicationContext`
- Create `@Configuration` class
- Add `@ComponentScan`
- Manage dependencies
- Decide how the app starts

This is **pure Spring Framework**.

### Above application using Spring Boot

#### Main Class

```
package com.testing.qe;
```

```
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.context.ApplicationContext;
```

```
@SpringBootApplication  
public class TestApplication {  
  
    public static void main(String[] args) {  
        ApplicationContext context = SpringApplication.run(TestApplication.class, args);  
        College college = context.getBean(College.class);  
        college.principalName();  
    }  
}
```

**No** `AnnotationConfigApplicationContext`

**No** `CollegeConfig` needed

## College class (almost same)

```
package com.testing.qe;

import org.springframework.stereotype.Component;

@Component
public class College {
    public void principalName() {
        System.out.println("Principal is Dr. Sharma");
    }
}
```

## What changed?

| Plain Spring          | Spring Boot  |
|-----------------------|--------------|
| Manual config class   | ✗ Not needed |
| Manual component scan | ✗ Auto       |
| XML / Java config     | ✗ Optional   |
| Embedded server       | ✗            |
| Boilerplate code      | ✗            |

## What `@SpringBootApplication` really doing?

`@SpringBootApplication`

This is **3 annotations combined**

`@Configuration`

`@EnableAutoConfiguration`

`@ComponentScan`

**Meaning:**

- `@Configuration` → Java config
- `@ComponentScan` → scans current package & subpackages
- `@EnableAutoConfiguration` → magic ✨ (auto setup)

## BENEFITS of Spring Boot

### ✓ 1. No configuration headache

- No XML
- No manual bean wiring
- No server setup

### ✓ 2. Auto-configuration (HUGE)

Example:

- Add `spring-boot-starter-web`
- BOOM ☐
  - Tomcat configured
  - DispatcherServlet created
  - JSON mapper ready

You didn't write **anything**.

### ✓ 3. Embedded server

No need to install:

- Tomcat
- JBoss
- WebLogic

Spring Boot ships with **embedded Tomcat**.

### ✓ 4. Production-ready

Spring Boot gives:

- Health checks
- Metrics
- Logging
- Externalized config

## How to EXECUTE Spring Boot app

### Option 1: From IDE (IntelliJ / Eclipse)

- Right-click on `TestApplication`
- Click **Run**

✓ Most common

### Option 2: Maven command

```
mvn spring-boot:run
```

### Option 3: Run JAR (production way)

```
mvn clean package java -jar target/test-0.0.1-SNAPSHOT.jar
```

## Project structure (Spring Boot way)

```
src/main/java
├── com.testing.qe
│   ├── TestApplication.java  <-- main class
│   └── College.java
```

**Main class must be at top package level**

(or component scanning will fail)

## Interview

Spring Boot = Spring + Auto Configuration + Embedded Server + Zero Boilerplate

## When should YOU use Spring Boot?

- ✓ REST APIs
- ✓ Microservices
- ✓ Real-world projects
- ✓ Faster development

## REST API

Instead of calling:

```
college.principalName();
```

We will expose it as a **REST endpoint**:

```
GET http://localhost:8080/college/principal
```

and get a response like:

```
Principal is Dr. Sharma
```

## Create a Spring Boot project

Use: IntelliJ / Eclipse **OR**

## Required dependency

Add only **Spring Web**

(Maven dependency)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

This is powered by **Spring Boot**, built on **Spring Framework**.

## Project structure (IMPORTANT)

```
src/main/java
├── com.testing.qe
│   ├── TestApplication.java ← main class
│   ├── College.java         ← business logic
│   └── CollegeController.java ← REST API
```

Main class must be at **top package level**.

## Main Application class

```
package com.testing.qe;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class TestApplication {

    public static void main(String[] args) {
        SpringApplication.run(TestApplication.class, args);
    }
}
```

✓ No `ApplicationContext`

✓ No config class

✓ No manual wiring

## Business logic (same as before)

```
College.java
```

```
package com.testing.qe;
```

```
import org.springframework.stereotype.Component;
```

```
@Component
public class College {

    public String principalName() {
        return "Principal is Dr. Sharma";
    }
}
```

✓ Spring manages this bean

✓ No `new College()` anywhere

## REST Controller (NEW)

CollegeController.java

```
package com.testing.qe;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
public class CollegeController {

    @Autowired
    private College college;

    @GetMapping("/college/principal")
    public String getPrincipalName() {
        return college.principalName();
    }
}
```

## What annotations mean (VERY IMPORTANT)

| Annotation             | Purpose                           |
|------------------------|-----------------------------------|
| @RestController        | Marks class as REST API           |
| @GetMapping            | Maps HTTP GET request             |
| @Autowired             | Injects <code>College</code> bean |
| @Component             | Makes class a Spring bean         |
| @SpringBootApplication | Auto config + scan                |

## How execution works (FLOW)

Browser / Postman

|  
| GET /college/principal  
v

CollegeController

|  
v

College (business logic)

|  
v

Response returned



## How to RUN the application

### ✓ Option 1: From IDE

- Right-click `TestApplication`
- Click **Run**

### ✓ Option 2: Maven

```
mvn spring-boot:run
```

### ✓ Option 3: JAR (production style)

```
mvn clean package java -jar target/*.jar
```

## How to TEST the REST API

### In browser

<http://localhost:8080/college/principal>

### In curl

```
curl http://localhost:8080/college/principal
```

## Why REST API is better than `main()` call

### Old way

Console output

Local JVM

Not reusable

Hard to scale

### REST way

HTTP response

Network accessible

Used by UI / Mobile

Cloud-ready

## Interview one-liner

**Spring Boot REST API exposes business logic using HTTP endpoints, making applications scalable, reusable, and platform-independent.**

## What next

- Convert response to **JSON**
- Add **POST API**
- Add **DTO**
- Connect to **Oracle DB**
- Explain **@RestController** vs **@Controller**